

Programming Theory: Dafny - tutorial

Sivakorn Lerttripinyo
Teaching Assistant 2025

Based on slides by Aleksandar Zeljić, Yacoub Hendi, and Axel Bergström

- 1 Outline
- 2 Introduction
- 3 The Dafny language
 - Syntax
 - Design by Contract
 - Printing
 - Assert
 - Quantifier
 - Loops
 - Functions
- 4 Installing and Running Dafny
- 5 Materials

Getting Started: Example

- `x` and `delta`, which are input parameters, are immutable.
- `res`, which is a return parameter, is mutable.
- You have to tell `method` what you want to return by using `returns`.
- There can be more than one return parameter.

Addition Method Without Pre and Post-Condition

```
method increase(x:int, delta:int) returns (res:int)
{
    // x := 10; // Cannot be done because x is immutable.
    res := x + delta;
}
```

Getting Started: Example

- `x` and `delta`, which are input parameters, are immutable.
- `res`, which is a return parameter, is mutable.
- You have to tell `method` what you want to return by using `returns`
- There can be more than one return parameter.

Addition Method Without Pre and Post-Condition (Error)

```
method increase(x:int, delta:int)
{
    var res := x + delta;
    return res // Error!
}
```

Getting Started: Example

- `x` and `delta`, which are input parameters, are immutable.
- `res`, which is a return parameter, is mutable.
- You have to tell `method` what you want to return by using `returns`
- There can be more than one return parameter.

Addition Method Without Pre and Post-Condition

```
method increase(x:int, delta:int) returns (res:int)
{
    return x + delta;
}
```

Quantifier

Dafny has universal and existential quantifiers:

forall $i :: bexpr \quad \forall i. bexpr$

exists $i :: bexpr \quad \exists i. bexpr$

Quantification over a range:

forall $i :: n \leq i < m ==> bexpr$

exists $i :: n \leq i < m \ \&\& \ bexpr$

Quantification over several variables:

forall $i, j :: n \leq i < j < m ==> bexpr$

exists $i, j :: n \leq i < j < m \ \&\& \ bexpr$

Quantifier - Example

- $\forall i : 1 \leq i \leq 3 : i^2 < 10$
 - forall $i :: 1 \leq i \leq 3 \implies i * i < 10$
 - $((1 \leq 1 \leq 3) \implies (1 * 1 < 10)) \wedge ((1 \leq 2 \leq 3) \implies (2 * 2 < 10)) \wedge ((1 \leq 3 \leq 3) \implies (3 * 3 < 10))$
- $\exists i : 1 \leq i \leq 3 : i^2 > 6$
 - exists $i :: 1 \leq i \leq 3 \ \&\& \ i * i > 6$
 - $((1 \leq 1 \leq 3) \wedge (1 * 1 > 6)) \vee ((1 \leq 2 \leq 3) \wedge (2 * 2 > 6)) \vee ((1 \leq 3 \leq 3) \wedge (3 * 3 > 6))$

Functions

- The function body must consist of exactly one **expression**, with the correct type.
 - Expression is something that produces a value.
 - Expression Example: `5+3`, `x*2`, `if x<0 then -x else x`
 - Statement is something that performs an action.
 - Statement Example
 - Declaring a variable: `var x := 5;`
 - Assignment: `x:=x+1;`
 - etc

Example of Function

```
function avg(a:int, b:int):int
{
    (a + b) / 2
}
```

Example of Invalid Function

```
function addOne(x:int):int
{
    var x := x+1; // This is an assignment statement.
    return y; // This is a return statement.
}
```

Functions

- **function DO NOT HAVE** a side-effect.
- Therefore, **function** can be used with the specification

Example of Valid Specification

```
function isPositive(x:int):bool
{
    x>0
}
method Good(arr:array<int>, x:int)
requires arr.Length > 0
requires isPositive(x)
modifies arr
ensures arr[0] == x
{
    arr[0] := x;
}
```

Array and Sequence

- Array is mutable. It requires **modifies** clause in methods.
- Sequence is immutable. Creating a modified sequence creates a new sequence instead.

Reminder: `old(v)` does not work with entire arrays, only individual values.

Example of using `old` Incorrectly

```
method BadOldExample(arr:array<int>)  
  requires arr.Length > 0  
  modifies arr  
  ensures arr == old(arr)  
{  
    arr[0] := 999;  
}
```

This program passes a verification, but it should not be since we change the array content.

Array and Sequence

Example of using old Correctly, but Failing the Verification

```
method GoodOldExampleFail(arr:array<int>)  
  requires arr.Length > 0  
  modifies arr  
  ensures arr[..] == old(arr[..])  
{  
    arr[0] := 999;  
}
```

This program fails a verification, but it should be since we change the array content.

Array and Sequence

Example of using old Correctly, and Passing the Verification

```
method GoodOldExamplePass(arr:array<int>)  
  requires arr.Length > 0  
  modifies arr  
  ensures arr[..] == old(arr[..])[0:=999]  
{  
    arr[0] := 999;  
}
```

This program now passes a verification again.